fortiss

Type-based Enforcement of Infinitary Trace Properties for Java

Chuangjie Xu (j.w.w. Ulirch Schöpp and Serdar Erbatur)

DDPD'21, 6-8 September 2021, Tallinn/Online



Enforcing Secure Programming Guidelines

- Does my Java program follow secure programming guidelines such as
  - *"All inputs must be sanitized."*
  - *"Any access to sensitive data must be authorized."*
  - *"Any access to sensitive data must be logged."*
  - *…*
- Can guidelines be verified continuously and incrementally?
- A lightweight tool for this can help programmers to avoid making typical errors during the development.



GuideForce[1]

GuideForce develops effect type systems for lightweight static analysis.

- Trace properties of programs
  - Functions of interests emit events, e.g.:
    `Server.login()` emits a *login* event; `Connection.close()` emits a *close* event
  - Each execution of a program generates a (finite or infinite) trace of events.
  - A guideline = a set of allowed event traces
- The type system has effect annotations to give information about the possible traces.
  - E.g., `login() ? readData() : close();` : `type & {`*login read*`,` *login close*`}`
- Type inference ⇒ effect computation
- If "effect ⊆ guideline", then the program adheres to the guideline.

[1] DFG project number 250888164

Example

Guideline 1: Any *access* to sensitive data occurs after *authorization*.

Server.java

```
16
17      void serve() {
18        while (hasQuery()) {
19          boolean authorized = verifyAuthorization();  // emits auth
20          if (authorized) {
21            readSensitiveData();  // emits access
22          }
23        }
24        logAccess();  // emits log
25      }
26
```

(this:EntryPoint, authorized:Base) & $\{auth, auth\ access\}^* \cdot \{auth\}$

(this:EntryPoint, authorized:Base) & $\{auth, auth\ access\}^*$

- Analysis Results
  - serve

This method adheres to the given guideline.

Terminating effect: $\{auth, auth\ access\}^* \cdot \{log\}$

Nonterminating effect: $\{auth, auth\ access\}^{\omega}$

Example

Guideline 2: Any *access* to sensitive data must be *logged*.

Server.java

```
16
17      void serve() {
18        while (hasQuery()) {
19          boolean authorized = verifyAuthorization(); // emits auth
20          if (authorized) {
21            readSensitiveData();  // emits access
22          }
23        }
24        logAccess(); // emits log
25      }
26
```

Generated trace: $(auth\ access)^{\omega}$

- Analysis Results
  - serve

This method may not satisfy the given guideline.

Terminating effect: $\{auth, auth\ access\}^{*} \cdot \{log\}$

Nonterminating effect: $\{auth, auth\ access\}^{\omega}$

Key Concepts

- **Effects**

  Capture information of terminating and nonterminating runs modularly in the type system.

  `verifyAuthorization() … & {auth} , ∅`
  `readSensitiveData() … & {access} , ∅`
  `LogAccess() … & {log} , ∅`
  `serve() … & {auth, auth access}* · {log} , {auth, auth access}`$^{\omega}$

- **Regions**

  Represent properties of values such as provenance information:

  `Null | CreatedAt(`l`) | Unknown | Tainted | Untainted | ...`

  Objects in different regions may be analyzed separately.

- **Abstraction**

  Finitely represents infinite languages to allow implementation of automatic type inference.

  E.g. *Hofmann and Chen*'s construction: finite words over {*auth*, *access*, *log*} ⇒ five equivalence classes

A Region-based Effect Type System

$$\Gamma \vdash e : T \text{ calls } S$$

- Γ is a typing context that maps program variables to regions.

  E.g., $\Gamma \equiv$ `x : CreatedAt(Example.java:21), s : Untainted, writer : Unknown`

A Region-based Effect Type System

$$\Gamma \vdash e : T \text{ calls } S$$

- Γ is a typing context that maps program variables to regions.
- T is a type-effect expression of the form $r_1 \,\&\, U_1 \mid \cdots \mid r_n \,\&\, U_n$.

  If e evaluates to a value, then the result is in region r_i and the event trace is in U_i for some i.

E.g., if e is the code on the right, then T could be

```
CreatedAt(Example.java:24) & {ab} |
CreatedAt(Example.java:26) & {b}
```

or even less precise type such as

```
Unknown & {ab,b}
```

```
21    Object x;
22    if (cond) {
23      emit(a);  \\ emits event a
24      x = new C();
25    } else {
26      x = new D();
27    }
28    emit(b);  \\ emits event b
29    return x;
```

A Region-based Effect Type System

$$\Gamma \vdash e : T \text{ calls } S$$

- Γ is a typing context that maps program variables to regions.
- T is a type-effect expression of the form $r_1 \,\&\, U_1 \mid \cdots \mid r_n \,\&\, U_n$.

  If e evaluates to a value, then the result is in region r_i and the event trace is in U_i for some i.
- S is a call expression of the form $U_1 \cdot \delta_1 \cup \cdots \cup U_m \cdot \delta_m$

  For any call that e makes, there is an i such that the event trace leading to the call is in U_i and the call goes to a method with signature δ_i.

  E.g., suppose `f()` has type Unknown & {a},

  then `f(); emit(b); g()` has the call expression $\{\varepsilon\} \cdot \delta_{\mathrm{f}} \cup \{\mathrm{ab}\} \cdot \delta_{\mathrm{g}}$

A Region-based Effect Type System

$$\Gamma \vdash e : T \text{ calls } S$$

- Γ is a typing context that maps program variables to regions.
- T is a type-effect expression of the form $r_1 \,\&\, U_1 \mid \cdots \mid r_n \,\&\, U_n$.

  If e evaluates to a value, then the result is in region r_i and the event trace is in U_i for some i.
- S is a call expression of the form $U_1 \cdot \delta_1 \cup \cdots \cup U_m \cdot \delta_m$

  For any call that e makes, there is an i such that the event trace leading to the call is in U_i and the call goes to a method with signature δ_i.

  Call expressions are for approximating the traces of nonterminating executions.

E.g., the call expression for the body of `h()` is $\{\mathrm{a}\} \cdot \delta_{\mathrm{h}}$.

The greatest solution of the equation $\delta_{\mathrm{h}} = \{\mathrm{a}\} \cdot \delta_{\mathrm{h}}$, namely $\{\mathrm{a}\}^{\omega}$, contains the trace generated by the nonterminating execution of `h()`.

```
void h() {
  emit(a);
  h();
}
```

Example



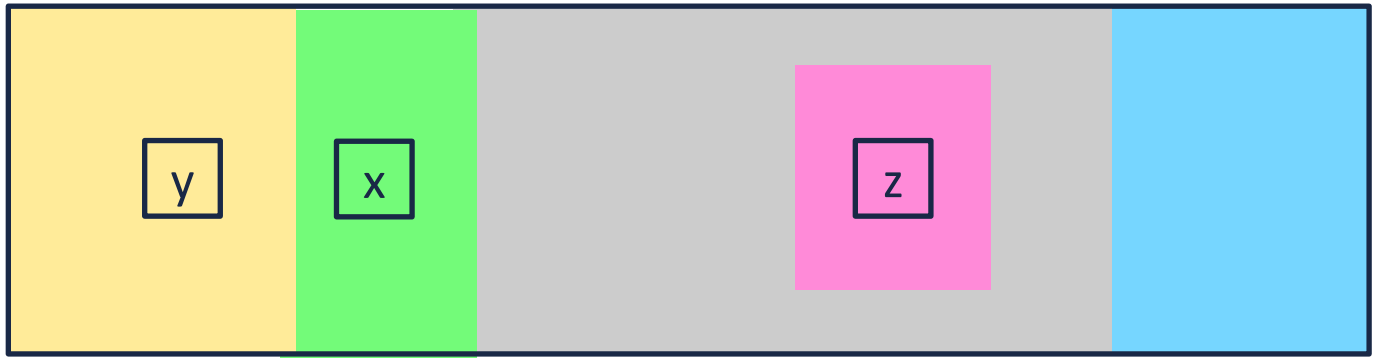
- If a method was analyzed without considering object information, then its effect should include the traces of all objects.

  E.g., `y.last()` and `z.last()` would have the same effect.

- Then the terminating method `linear()` would have the same effect of the nonterminating method `cyclic()`.

```
1   class Node {
2     Node next;
3     Node last() {
4       emit(a);
5       if (next == null) {
6         return this;
7       } else {
8         return next.last();
9       }
10    }
11  }
12
13  Class Test {
14    Node linear() {
15      Node x = new Node();
16      Node y = new Node();
17      y.next = x;
18      return y.last();
19    }
20    Node cyclic() {
21      Node z = new Node();
22      z.next = z;
23      return z.last();
24    }
25  }
```

Example



- If a method was analyzed without considering object information, then its effect should include the traces of all objects.

  E.g., `y.last()` and `z.last()` would have the same effect.

- Then the terminating method `linear()` would have the same effect of the nonterminating method `cyclic()`.

- To improve the precision of effect typing, we use regions to narrow down referenced objects.

  Objects in different regions are analyzed separately.

```
1   class Node {
2     Node next;
3     Node last() {
4       emit(a);
5       if (next == null) {
6         return this;
7       } else {
8         return next.last();
9       }
10    }
11  }
12
13  Class Test {
14    Node linear() {
15      Node x = new Node();
16      Node y = new Node();
17      y.next = x;
18      return y.last();
19    }
20    Node cyclic() {
21      Node z = new Node();
22      z.next = z;
23      return z.last();
24    }
25  }
```

Example

- Write r_x = CreatedAt(Node.java:15)
  r_y = CreatedAt(Node.java:16)
  r_z = CreatedAt(Node.java:21)

  Let δ_r denote the signature of the method `last()` at region r.

- `last()` is given different types depending on the region:

  class Node@Null
    last() : ∅ calls ∅

  class Node@r_x
    last() : r_x & {a} calls {a} · δ_{Null}

  class Node@r_y
    last() : r_x & {aa} | r_y & {a} calls {a} · δ_{r_x}

  class Node@r_z
    last() : r_z & {a}⁺ calls {a} · δ_{r_z}

- The two methods in Test are typed as follows:

  linear() : r_x & {aa} | r_y & {a} calls δ_{r_y}

  cyclic() : r_z & {a}⁺ calls δ_{r_z}

```
1   class Node {
2     Node next;
3     Node last() {
4       emit(a);
5       if (next == null) {
6         return this;
7       } else {
8         return next.last();
9       }
10    }
11  }
12
13  Class Test {
14    Node linear() {
15      Node x = new Node();
16      Node y = new Node();
17      y.next = x;
18      return y.last();
19    }
20    Node cyclic() {
21      Node z = new Node();
22      z.next = z;
23      return z.last();
24    }
25  }
```

Example

- Write r_x = CreatedAt(Node.java:15)
  r_y = CreatedAt(Node.java:16)
  r_z = CreatedAt(Node.java:21)

  Let δ_r denote the signature of the method `last()` at region r.

- `last()` is given different types depending on the region:

  class Node@Null
    last() : ∅ , ∅

  class Node@r_x
    last() : r_x & {a} , ∅

  class Node@r_y
    last() : r_x & {aa} | r_y & {a} , ∅

  class Node@r_z
    last() : r_z & $\{a\}^+$, $\{a\}^\omega$

- The two methods in Test are typed as follows:

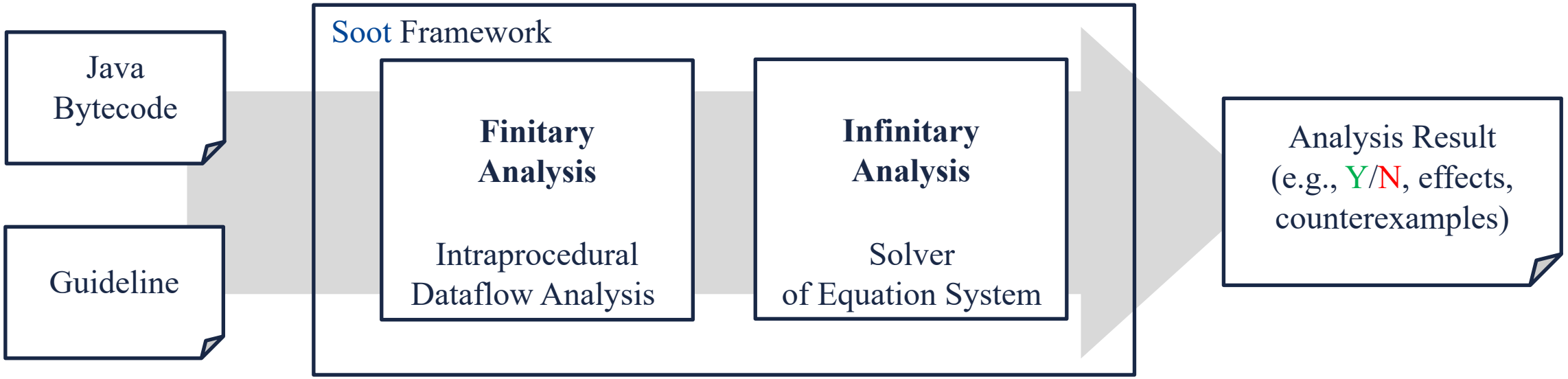
  linear() : r_x & {aa} | r_y & {a} , ∅

  cyclic() : r_z & $\{a\}^+$, $\{a\}^\omega$

```
1   class Node {
2     Node next;
3     Node last() {
4       emit(a);
5       if (next == null) {
6         return this;
7       } else {
8         return next.last();
9       }
10    }
11  }
12
13  Class Test {
14    Node linear() {
15      Node x = new Node();
16      Node y = new Node();
17      y.next = x;
18      return y.last();
19    }
20    Node cyclic() {
21      Node z = new Node();
22      z.next = z;
23      return z.last();
24    }
25  }
```

Summary

- We introduce a new effect type system for (Featherweight) Java to analyze trace properties of both terminating and *non-terminating* programs.
  - simpler – class information can be omitted
  - more precise – effect is captured separately at each region, i.e., $r_1 \;\&\; U_1 \mid \cdots \mid r_n \;\&\; U_n$
- We have a prototype implementation of type inference based on the Soot framework, available at: https://github.com/cj-xu/GuideForceJava



Thank you!!